

DISEÑO DE PATRONES – SEMANA 09

Ejercicios Propuestos y Resueltos – Decorator y Composite

Decorator

Diseñe un sistema de notificaciones donde la clase base envíe mensajes por correo electrónico y, mediante el uso del patrón Decorator, se puedan añadir dinámicamente nuevas funcionalidades como el envío de SMS y WhatsApp, de modo que el mismo mensaje pueda transmitirse por múltiples canales sin modificar el código original de la clase base.

```
interface Notificador
{
    void enviar(String mensaje);
}

class NotificadorEmail implements Notificador
{
    public void enviar(String mensaje)
    {
        System.out.println("Enviando correo: " + mensaje);
    }
}

class NotificadorDecorator implements Notificador
{
    protected Notificador notificador;

    public NotificadorDecorator(Notificador notificador)
    {
        this.notificador = notificador;
    }

    public void enviar(String mensaje)
    {
        notificador.enviar(mensaje); // delega la acción
    }
}

class NotificadorSMS extends NotificadorDecorator
{
    public NotificadorSMS(Notificador notificador)
    {
        super(notificador);
    }

    public void enviar(String mensaje)
    {
        super.enviar(mensaje);
        System.out.println("Enviando SMS: " + mensaje);
    }
}
```

DISEÑO DE PATRONES – SEMANA 09

Ejercicios Propuestos y Resueltos – Decorator y Composite

```
class NotificadorWhatsApp extends NotificadorDecorator
{
    public NotificadorWhatsApp(Notificador notificador)
    {
        super(notificador);
    }
    public void enviar(String mensaje)
    {
        super.enviar(mensaje);
        System.out.println("Enviando WhatsApp: " + mensaje);
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Notificador notificador = new NotificadorWhatsApp(new NotificadorSMS(
            new NotificadorEmail()));
        notificador.enviar("Reunión a las 5 PM");
    }
}
```

Composite

El sistema debe permitir representar un menú jerárquico de restaurante donde tanto platos individuales como menús completos se traten de manera uniforme, definiendo una interfaz común llamada MenuComponent que declare la operación de mostrar, implementando clases hoja (Plato) que representan cada plato y muestran su nombre, y clases compuestas (Menu) que almacenan y gestionan una colección de componentes delegando la operación en cada hijo y añadiendo la lógica de mostrar el título del menú, de modo que el cliente pueda invocar la operación sin distinguir si trabaja con un plato individual o con un menú completo.

```
import java.util.ArrayList;
import java.util.List;

interface MenuComponent
{
    void mostrar();
}

class Plato implements MenuComponent
{
    private String nombre;

    public Plato(String nombre)
    {
        this.nombre = nombre;
    }
}
```

DISEÑO DE PATRONES – SEMANA 09

Ejercicios Propuestos y Resueltos – Decorator y Composite

```
}

    public void mostrar()
    {
        System.out.println("Plato: " + nombre);
    }
}

class Menu implements MenuComponent
{
    private String nombre;
    private List<MenuComponent> items = new ArrayList<>();
    public Menu(String nombre)
    {
        this.nombre = nombre;
    }
    public void add(MenuComponent item)
    {
        items.add(item);
    }
    public void remove(MenuComponent item)
    {
        items.remove(item);
    }
    public void mostrar()
    {
        System.out.println("Menú: " + nombre);
        for (MenuComponent item : items)
        {
            item.mostrar();
        }
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Plato sopa = new Plato("Sopa de verduras");
        Plato pasta = new Plato("Pasta al pesto");
        Plato postre = new Plato("Tiramisú");
        Menu menuPrincipal = new Menu("Menú del día");
        menuPrincipal.add(sopa);
        menuPrincipal.add(pasta);
        menuPrincipal.add(postre);
        menuPrincipal.mostrar();
    }
}
```